Automated Secure Alarm Protocol (ASAP)

Application Programming Interface (API) Documentation

Version 1.4

The information contained in this document is classified Mission Critical Partners (MCP) and The Monitoring Association (TMA) CONFIDENTIAL PROPRIETARY. The use and divulgence of any part of this information can seriously affect the welfare and financial security of the company. The distribution of information to any individual not employed by MCP/TMA or to any organization is strictly prohibited. Any MCP/TMA employee receiving this document will be held strictly accountable for the discretion and judgment associated with the use of this information.

Table of Contents

Purpose 1

Use Cases 1

AWS Hosted ASAP API Architecture 1

- Monitoring Center (Central Station) API Security 2
- CAD/ECC API Security 2

AWS Hosted ASAP Interface Requirements 2

AWS Hosted ASAP API Specification 3

- Monitoring Center (Central Station) Interface 3
  - Path 1: Default Behavior (mTLS) 3
- CAD/ECC Interfaces 4
  - Path 1: Default Behavior (OAuth2) 4
  - Path 2: Device Attestation/mTLS 5
  - Path 3: Zero Trust Gateway 5
- Message Payload 6
  - APCO Standards 6

Endpoint Interface Requirements 7

Environment Servers 8

Security 8

- OAuth2 8
  - Authentication Server API 9
- mTLS 10
- OAuth2 mTLS Client Authentication 10
- Efficiency Considerations 11
- Bearer Token Caching 11

Appendix A: ASAP Key Element FAQs 12

Appendix B: OpenAPI Specification for ASAP 14

Appendix C: Sample API for an ECC CAD 17

Appendix D: Basic Troubleshooting Return Codes 19

Monitoring Center (Central Station) to ASAP Interface 19
Emergency Call Center using OAuth to ASAP Interface (path 1) 19
Emergency Call Center using mTLS to ASAP Interface (path 2) 20
Emergency Call Center using Zero Trust Gateway to ASAP Interface (path 3) 20
Emergency Call Center OAuth2 Token Interface 20
Appendix E: Verifying API Connectivity 22
OAuth Tests 22
Step 1: Request Access Token 22
Step 2: Send Message 24
MTLS Tests 25
Monitoring Center API 25
MTLS ECC API 26

Figure 1 API Architecture 2
Figure 2 OAuth Flow 9

Document History

Date	Version	Editor	Description
8-April-25	1.0	Philip Hartman	First Draft
24-April-25	1.1	Philip Hartman	Refine Security Summary Add trouble shooting guidance Define Paths (MC<->Broker<->ECC)
23-May-25	1.2	Joe Mierwa	Add test code examples
09-Sept-25	1.3	Charisma Petrie	Add MASterMind information to Message Payload section
29-Oct-25	1.4	Charisma Petrie	Replaced references to “MASterMind” with “monitoring center”.

Purpose

The purpose of the Automated Secure Alarm Protocol (ASAP) application programming interface (API) document is to outline a guide for implementers responsible for interfacing either an emergency communications center (ECC) and its computer-aided dispatch (CAD) system or alarm monitoring center with the Amazon Web Services (AWS) hosted ASAP broker to facilitate the ASAP exchange. ASAP is property of The Monitoring Association (TMA).

The document is organized with the details of the API first, followed by background details explaining how to effectively use and interate an API, including its functionality, endpoints, data formatrs, and authentication protocols.

Use Cases

ECCs currently receiving alarm messages through the National Law Enforcement Telecommunications Systems (Nlets) hosted ASAP broker will have the option of moving to the AWS hosted ASAP broker. While the message payload remains the same, using the AWS hosted ASAP broker allows the ECC to receive messages directly through the ASAP API, which does not require Nlets and the state switches to connect directly to the ECC's CAD system.

Monitoring centers currently using Nlets hosted ASAP (2.0) will be able to connect to the AWS hosted ASAP broker.

AWS Hosted ASAP API Architecture

The Association of Public-Safety Communications Officials International (APCO)/TMA American National Standard (ANS) 2.101.3-2021[1] defines the messaging required between a monitoring agency and an ECC; how the messages are communicated between the endpoints is undefined. A study of the messages themselves implies a duplex message exchange pattern is required to implement ASAP effectively between two endpoints. To this end, the ASAP architecture implements this pattern and is illustrated below using an event-driven REST API architecture.

[1] *Alarm Monitoring Company to Emergency Communications Center (ECC) Computer-Aided Dispatch (CAD) Automated Secure Alarm Protocol (ASAP)*

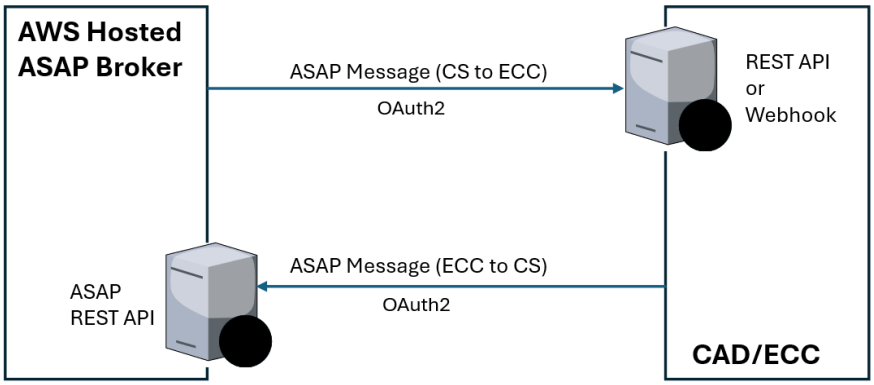


Figure 1 API Architecture

The above diagram shows the connection between ASAP and an ECC. Note that both sides must host a service to be able to receive incoming messages. The event-driven REST API architecture is achieved with a rest API whose only allowed HTTP methods are a "POST."

Monitoring Center (Central Station) API Security

The API gateway messaging relies on the use of mutual Transport Layer Security (mTLS). This security mechanism is mandated by TMA and relies on each end verifying that the sender is who they say they are using X.509 certificates. Further, each side uses the received certificate serial number to confirm that the received message came from an IP address that is associated with the domain name that has a preconfigured association between a certificate serial number and a domain name.

CAD/ECC API Security

The API gateway messaging relies on the use of Open Authentication 2.0 (OAuth2) between the AWS hosted ASAP and ECC endpoints. Additional details regarding OAuth2 are provided later in this document.

AWS Hosted ASAP Interface Requirements

The APCO ASAP specification implies the existence of bi-directional messaging between a monitoring center (central station or CS) and an ECC. This means that each endpoint, CS, or ECC must be capable of sending and receiving messages. Within the context of ASAP, this means that ECC endpoints directly connected to ASAP as well as CS:

1. Must be able to send ASAP messages to ASAP using the API specification described in this document.

2. Must be able to implement and deploy an HTTP-based service to receive messages. This service can be either a REST or Webhook service and must have an API conforming to the minimum requirements for endpoints described in this document.

AWS Hosted ASAP API Specification

The API specification presented here is from the perspective of ASAP. As such, it presents the APIs necessary for both CSs and ECCs to send messages. Note that the formal OpenAPI specification described here is included at the end of this document.

Monitoring Center (Central Station) Interface

Path 1: Default Behavior (mTLS)

Method	Path
POST	**/asap/central-station**

This handles ASAP messages received from all CSs and uses mTLS as the primary authentication mechanism.

Parameters: There are no parameters associated with this API call.
Request Body: application/xml
Security: "mutualTLS"

Responses

Code	Description	
200	Successful response	
400	Bad request	There is an error with the payload of the message.
401	Unauthorized	There was an unexpected error with the authentication process that could be related to the mTLS handshake or the secondary authorization as described below in '403'.

Code	Description	
403	Forbidden	The IP address from which the message originated is not an IP address associated with the host name specified in the client certificate.
500	Internal server error	Authorization/Authentication passed but an unexpected error occurred in processing the request. Notify ASAP Support.

CAD/ECC Interfaces

Path 1: Default Behavior (OAuth2)

Method	Path
POST	**/asap/ecc**

This handles all ASAP messages received by ECCs (CAD systems) using OAuth2 as an authentication mechanism. While it is not the most secure method, as it implements half of what is recommended for zero-trust architectures, timeframes and most CAD vendors prefer this authentication mechanism.

Parameters: There are no parameters associated with this API call.
Request Body: application/xml
Security: "OAuth2"

Responses

Code	Description	
200	Successful response	
400	Bad request	There is an error with the payload of the message.
401	Unauthorized	These are problems with the client ID, client secret, or redirect URI.
403	Forbidden	The client has insufficient permissions to access the resource.
500	Internal server error	Authorization/Authentication passed but an unexpected error occurred in processing the request. Notify ASAP Support.

Path 2: Device Attestation/mTLS

Method	Path
POST	**/asap/ecc/da**

This handles ECC ASAP messages using mTLS as the primary authorization mechanism. Note that “da” in the path is short for "device attestation," meaning that the device identity is verified using mTLS.

Parameters: There are no parameters associated with this API call.
Request Body: application/xml
Security: “mutualTLS”

Responses

Code	Description	
200	Successful response	
400	Bad request	There is an error with the payload of the message.
401	Unauthorized	There was an unexpected error with the authentication process that could be related to the mTLS handshake or the secondary authorization as described below in ‘403’.
403	Forbidden	The IP address from which the message originated is not an IP address associated with the host name specified in the client certificate.
500	Internal server error	Authorization/Authentication passed but an unexpected error occurred in processing the request. Notify ASAP Support.

Path 3: Zero Trust Gateway

Method	Path
POST	**/asap/ecc/ztg**

This handles ECC ASAP messages using both OAuth2 for credential authentication and mTLS for device level authentication. The path acronym 'ztg' is short for "zero trust gateway", using continuous verification. This API will not be implemented in the short term unless any CAD vendors connecting to ASAP require it. It is possible that some ECCs may require their CAD vendors to support this.

Parameters: There are no parameters associated with this API call.
Request Body: application/xml
Security: "mutualTLS" and "OAuth2"

Responses

Code	Description	
200	Successful response	
400	Bad request	There is an error with the payload of the message.
401	Unauthorized	There was an unexpected error with the authentication process that could be related to the mTLS handshake or the secondary authorization as described below in '403'.
403	Forbidden	The IP address from which the message originated is not an IP address associated with the host name specified in the client certificate.
500	Internal server error	Authorization/Authentication passed but an unexpected error occurred in processing the request. Notify ASAP Support.

Message Payload

APCO Standards

The payloads contained in the request body of all messages passed through the broker are required to support the APCO/TMA ANS 2.101.3-2021 standard, which builds on the 2014 standard, as outlined in https://www.apcointl.org/~documents/standard/21013-2021-asap-to-psap/?layout=file.

In addition to the other materials provided to you regarding migrating to a direct Rest connection with the new ASAP broker, there are a few considerations you will need to account for when

Receiving requests from any monitoring center:

1. At this time, monitoring centers only support Version 3.3 of the ASAP (Automated Secure Alarm Protocol) and as such, any responses to alarms, updates or address verifications must be conformant to Version 3.3.

2. Some monitoring centers that have implemented the ASAP standard to use hardcoded XML element prefixes illustrated in the standard examples when parsing received messages. The ASAP message broker does not manipulate the payloads of any messages. Therefore, all ASAP messages sent to monitoring centers must ensure that the following prefixes are used for the associated namespaces:

    - em: http://niem.gov/niem/domains/emergencyManagement/2.0
    - j: http://niem.gov/niem/domains/jxdm/4.0
    - s: http://niem.gov/niem/structures/2.0
    - nc: http://niem.gov/niem/niem-core/2.0
    - apco-alarm: http://www.apcointl.com/new/commcenter911/external-alarm.xsd

Endpoint Interface Requirements

Every endpoint that will be communicating with ASAP must implement and deploy an HTTP-based API that:

1. Uses a POST method
2. Does not use parameters
3. Uses a request body of “application/XML”
4. Uses a security authentication mechanism of OAuth2, mTLS, or both

Historically, CSs currently using ASAP have all implemented their REST services with APIs using mTLS. This usage has been mandated by TMA. In contrast, CAD vendors have

expressed the desire to use services employing OAuth2 as the primary security authentication mechanism.

Environment Servers

The ASAP API gateway will operate in three environments—Production, Certification, and Test.

Production: https://nexus.tma-asap.org

Certification: https://cert.nexus.tma-asap.org

Test: https://api-test-dsis-mcp.com

The ASAP authentication server, for use with OAuth2, will also operate in the same three environments as the API gateway.

Production: https://to.be.determined.org

Certification: https:// to.be.determined.org

Test: https:// to.be.determined.com

Security

The "never trust, always verify" model focuses on protecting resources by verifications of users before access is granted. The ASAP API gateway supports both mTLS and OAuth2 as security mechanisms. For an API gateway to be in line with a zero-trust architecture, both security mechanisms need to be used on the API. While the gateway can support it, it will not be configured until the vendors associated with the ASAP API are willing to implement both security mechanisms.

OAuth2

OAuth2 is an open-standard authorization protocol that allows third-party applications to obtain limited access to a user's protected resources without sharing the user's credentials. It works by issuing access tokens through an authorization server, which the client application uses to access the resources hosted on a resource server. In ASAP, the authorization server is hosted using AWS Cognito. OAuth2 is the default authentication method for ASAP. An example of its usage is shown below.

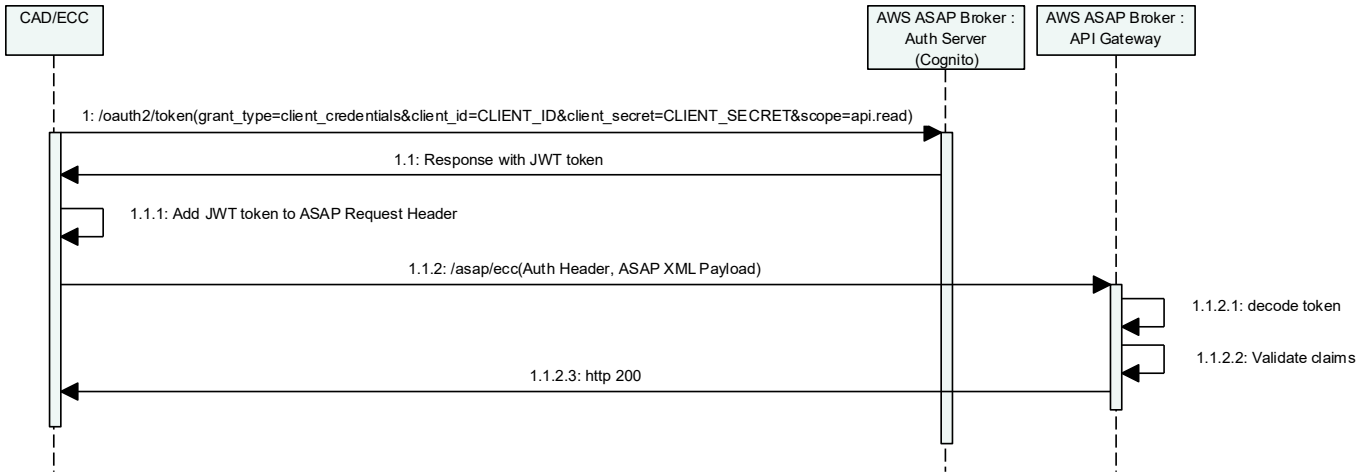


Figure 2 OAuth Flow

Authentication Server API

Method	Path
POST	**/oauth2/token**

This retrieves a token to be placed in the header of the relevant ASAP API invocation.

Parameters: There are no parameters associated with this API call.
Request Body: application/x-www-form-urlencoded header

The format of the Request Body is a "&" delimited string of name/value pairs of the form

<parameterName>=<value>

where the parameters being sent in the request are:

1. **grant_type=client_credentials**:
    - Specifies the type of OAuth2 flow being used
    - For machine-to-machine communication, this is always set to client_credentials
2. **client_id=CLIENT_ID**:
    - Unique identifier for the client application
    - Issued by the authorization server when the client is registered
3. **client_secret=CLIENT_SECRET**:

- The secret key associated with the client ID
- Used to authenticate the client application to the authorization server

4. **scope= api/api-access**:
    - Specifies the permissions (scopes) being requested by the client. For the API Gateway in ASAP Nexus, it is always “api/api-access”.

An example of the above in the Request Body is:

> “grant_type=client_credentials&client_id=CLIENT_ID&client_secret=CLIENT_SECRET&scope=api/api-access”

Response

A successful response to an oauth2/token post is a simple JSON object containing:

"access_token"	The actual token that the client application uses to authenticate itself when making requests to the protected API.
"token_type"	“Bearer”
"expires_in"	The lifetime of the access token, measured in seconds.
"scope":	"api/api-access"

mTLS

For any API path using mTLS, the authentication is twofold:

1. The mTLS handshake sends the client public certificate to the server, which the server verifies.
2. After that is completed, the serial number is retrieved from the certificate and used to access the DNS host name that was entered as part of the registration process in ASAP View. The hostname then checks to see if the source IP address of the HTTP request is from an IP address associated with the DNS.

This verifies the client requesting the service is a valid certificate and also assures that the certificate is being sent from a machine in the domain for which it was issued.

OAuth2 mTLS Client Authentication

This authentication mechanism is defined by the Internet Engineering Task Force (IETF) RFC 8705[2]. It replaces the use of the client secret mechanism, used by the standard OAuth2, with the use of mTLS for the authentication server call to link that client certificate to a JSON Web Token (JWT). When the application API is called, also using mTLS for the

[2] *OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens*

gateway call, the JWT certificate information validates that the client that sent the API request is also the client that sent the message. In short, it is a much stronger layer of security.

Efficiency Considerations

For an implementer writing an OAuth2 authenticated service that the ASAP will connect to, defining the token lifetime will impact the level of security.

In a zero-trust architecture, the effectiveness of continuous scanning (or continuous verification) is inversely related to the token lifetime because short-lived tokens reduce the attack window by limiting how long a compromised token can be used. With shorter token lifetimes, the system must frequently revalidate or reissue tokens, whereas a longer token lifetime may improve the performance of the system at the expense of making continuous scanning less effective.

It is important to find a balance between performance and security when setting the token life span.

Bearer Token Caching

Bearer tokens are credentials that grant access to protected resources. Once issued, they are typically cached on the client side in memory or local storage for reuse across requests.

For implementers building clients to communicate with ASAP, it is important to implement a lifecycle process to get, reuse, and replace your bearer tokens. This will streamline the amount of traffic coming and going from the API service and will ensure that user credentials are rarely exposed in the traffic; even if the traffic were to be intercepted and somehow decrypted, the bad actor would likely only receive a bearer token, which has already expired during the decryption window, rather than obtaining user credentials.

In zero-trust architecture, bearer token caching introduces both security challenges and potential tradeoffs with usability that you need to consider. For example, bearer token caching with a limited lifetime will balance the accessibility of the system against limiting the level of any breach.

Appendix A: ASAP Key Element FAQs

What secure information interface exchange does ASAP use?

ASAP uses the **RESTful API** interface, which is secure, flexible, and efficient.

What return format does ASAP use?

ASAP uses the **XML** file format to store, transmit, and reconstruct data.

What reference tokens does ASAP use?

ASAP uses **JWT bearer tokens** for authentication. A JWT bearer token is a very compact, printable representation of a series of claims, along with a signature to verify its authenticity.

What is Zero-trust Architecture?

Zero-trust Architecture (ZTA) is a modern security framework designed to eliminate implicit trust in users, devices, and systems, regardless of their location within or outside an organization's network. It is used in environments where security is paramount, such as the finance, health care and Criminal Justice Information Services (CJIS). Unlike traditional perimeter-based security models, ZTA operates on the principle of "never trust, always verify." It continuously authenticates and authorizes every access request based on dynamic factors such as user identity, device posture, and contextual risk.

Core Principles of ZTA

- **Verify Explicitly**: Always authenticate and authorize access based on multiple factors, such as identity, device health, location, and behavior.
- **Use Least Privilege Access**: Grant users and devices the minimum permissions necessary to perform their tasks, reducing the potential impact of a breach.
- **Assume Breach**: Operate under the assumption that threats exist both inside and outside the network. Continuously monitor and validate access throughout a session to detect and contain malicious activity.

What is Event-driven Architecture?

Event-driven Architecture (EDA) is a software design pattern where systems detect, produce, and respond to events—significant changes in state—either in real-time or near real-time. It enables decoupled communication between components, such as microservices, by using events as the central mechanism for interaction.

EDA promotes scalability, agility, and fault tolerance by allowing independent development and deployment of services.

Key Components of EDA

- **Event Producers**: Generate events (e.g., a user adding an item to a shopping cart).
- **Event Routers**: Filter and distribute events to consumers.
- **Event Consumers**: React to events asynchronously, performing actions like updates or notifications.

Appendix B: OpenAPI Specification for ASAP

This is the formal OpenAPI specification governing the ASAP API gateway. For implementers wanting to use tooling to generate the client or server code needed to support this API, it is recommended that this API be redacted to include only the desired path. For instance, a CAD vendor wanting to use the OAuth2 would delete paths '/asap/central-station', '/asap/ecc/da' and '/asap/ecc/ztg'.

Some tooling for OpenAPI scripts cannot support OpenAPI 3.1.x. As a result, the version of the below text needs to be changed and all references to the "ClientCertificateAuth" need to be removed.

```
openapi: 3.1.0
info:
  title: TMA ASAP Alarm Signal Traffic Manager (ASAP ASM) API
  version: 1.0.0-oas3.1
  description: >
    This API accepts XML payloads and relies on preprocessing
components for
    message handling. Maintains compatibility with TMA 2.0 ASAP
message broker
    implementation.
servers:
  - url: https://api-test.dsis-mcp.com
    description: The URL for AWS ASAP Broker in the test environment
  - url: https://api-dev.dsis-mcp.com
    description: The URL for AWS ASAP Broker in the dev environment
  - url: https://tbd.tma-asap.org
    description: The URL for AWS ASAP Broker in the certification
environment
  - url: https://tbd.tma-asap.org
    description: The URL for AWS ASAP Broker in the production
environment
paths:
  /asap/central-station:
    post:
      summary: Handle ASAP messages from Central Stations
      operationId: handleCentralStationRequest
      requestBody:
        required: true
        content:
          application/xml:
            schema: {}
      responses:
        '200':
          description: Successful response
        '400':
          description: Bad request
        '500':
```

```
          description: Internal server error
      security:
        - ClientCertificateAuth: []
  /asap/ecc:
    post:
      summary: Handle ECC ASAP messages (OAuth2 Protected, which is
the default)
      operationId: handleECCRequestOAuth
      requestBody:
        required: true
        content:
          application/xml:
            schema: {}
      responses:
        '200':
          description: Successful response
        '400':
          description: Bad request
        '415':
          description: Unsupported media type
        '500':
          description: Internal server error
      security:
        - OAuth2:
            - write:asap
  /asap/ecc/da:
    post:
      summary: >-
        Handle ECC ASAP messages (where da = "Device Attestation",
meaning that
        the device identity is verified using mTLS)
      operationId: handleECCRequestDeviceIdentity
      requestBody:
        required: true
        content:
          application/xml:
            schema: {}
      responses:
        '200':
          description: Successful response
        '400':
          description: Bad request
        '415':
          description: Unsupported media type
        '500':
          description: Internal server error
      security:
        - ClientCertificateAuth: []
  /asap/ecc/ztg:
    post:
      summary: >-
```

```
        Handle ECC ASAP messages (where ztg = "Zero Trust Gateway"
using
        Continuous Verification, which is OAuth2 and mTLS
authentication)
      operationId: handleECCRequestZeroTrust
      requestBody:
        required: true
        content:
          application/xml:
            schema: {}
      responses:
        '200':
          description: Successful response
        '400':
          description: Bad request
        '415':
          description: Unsupported media type
        '500':
          description: Internal server error
      security:
        - OAuth2:
            - write:asap
          ClientCertificateAuth: []
components:
  securitySchemes:
    OAuth2:
      type: oauth2
      description: >
        Client Credentials flow for machine-to-machine communication.
        Short-lived tokens limit exposure from potential interception
[2][4].
      flows:
        clientCredentials:
          tokenUrl: https://auth.tma-asap.org/oauth2/token
          scopes:
            write:asap: Allows sending ASAP messages
    ClientCertificateAuth:
      type: mutualTLS
      description: >
        X.509 certificate authentication ensures strong device
identity proof.
        Certificates are managed through enterprise PKI infrastructure
[10].
  schemas:
    Empty:
      type: object
```

Appendix C: Sample API for an ECC CAD

This is a sample OpenAPI specification supporting incoming ASAP messages from the ASAP API gateway. This sample is written according to the OpenAPI specification 3.0 and is supported in most tooling to auto-generate the service. It is provided to begin the development to enable a CAD system to be ASAP-capable.

As mentioned elsewhere in this document, if you wish to support mTLS or desire increased alignment with zero-trust architecture, you can modify the defined path content using the ASAP gateway as a guide.

```
openapi: 3.0.0
info:
  title: TMA ASAP CAD/ECC Alarm Signal Emergency Communication Center
API
  version: 1.0.0
  description: >
    This sample API accepts ASAP XML payloads sent from either the AWS
ASAP Broker or
    directly from a Monitoring Center/Central Station. This is to help
guide the
    development of an HTTP based API by CAD vendors.
    Maintains compatibility with TMA 2.0 ASAP message broker
implementation.

servers:
  # All endpoints enforce HTTPS to prevent MITM attacks and ensure
encrypted communication
  - url: https://test-ecc.asap.com

paths:
  /asap-event:
    post:
      summary: Handle incoming ECC ASAP messages (OAuth2 Protected)
      operationId: handleECCEvent
      requestBody:
        required: true
        content:
          application/xml:
            schema: {}
      responses:
        '200':
          description: Successful response
        '400':
          description: Bad request
        '415':
          description: Unsupported media type
        '500':
          description: Internal server error
```

```
      security:
        - OAuth2: ['write:asap']  # Fine-grained scope limits token
privileges

components:
  securitySchemes:
    OAuth2:
      type: oauth2
      description: >
        Client Credentials flow for machine-to-machine communication.
        Short-lived tokens limit exposure from potential interception.
      flows:
        clientCredentials:  # Recommended for backend services
          tokenUrl: https://auth.tma-asap.org/oauth2/token  # HTTPS
enforced for token exchange
          scopes:
            write:asap: Allows processing received  ASAP messages  #
Explicit scope definition prevents scope injection
  schemas:
    Empty:
      type: object
```

Appendix D: Basic Troubleshooting Return Codes

This appendix provides very high-level trouble shooting tips for codes returned by ASAP to a monitoring center of emergency call center.

Monitoring Center (Central Station) to ASAP Interface

Code	Description	Suggested Troubleshooting Steps
400	Bad request	Check for correct endpoint Check the request syntax Check for invalid characters or incorrect data types Ensure all required fields are present. Consider using a debugging tool like Postman.
401	Unauthorized	Check server certificate validation. Check correct client certificate chain.
403	Forbidden	The IP address from which the message originated is not an IP address associated with the host name specified in the client certificate.
500	Internal server error	Contact the ASAP Support Team.

Emergency Call Center using OAuth to ASAP Interface (path 1)

Code	Description	Suggested Troubleshooting Steps
400	Bad request	Check for correct endpoint Check the request syntax Check for invalid characters or incorrect data types Ensure all required fields are present. Consider using a debugging tool like Postman.
401	Unauthorized	Ensure the correct client ID is used. Check that the correct client secret is used Check the redirect URI or callback URL is correct.
403	Forbidden	Review the OAuth consent screen to make sure the appropriate scope is enabled.

Code	Description	Suggested Troubleshooting Steps
500	Internal server error	Contact the ASAP Support Team.

Emergency Call Center using mTLS to ASAP Interface (path 2)

Code	Description	Suggested Troubleshooting Steps
400	Bad request	Check for correct endpoint Check the request syntax Check for invalid characters or incorrect data types Ensure all required fields are present. Consider using a debugging tool like Postman.
401	Unauthorized	There was an unexpected error with the authentication process that could be related to the mTLS handshake or the secondary authorization as described below in '403'.
403	Forbidden	The IP address from which the message originated is not an IP address associated with the host name specified in the client certificate.
500	Internal server error	Contact the ASAP Support Team.

Emergency Call Center using Zero Trust Gateway to ASAP Interface (path 3)

Suggested troubleshooting steps will be included upon the first ECC request for Zero Trust Gateway.

Emergency Call Center OAuth2 Token Interface

Code	Description	Suggested Troubleshooting Steps
400	Bad request	The request was malformed or missing required parameters. This is the most common error for invalid or incomplete requests, such as: • Missing or invalid `grant_type` • Invalid client credentials

		• Invalid or missing scope • Unsupported grant type • Other parameter errors
401	Unauthorized	The client failed to authenticate (e.g., wrong client ID/secret, invalid Authorization header). This is returned when authentication fails or credentials are missing/incorrect.
403	Forbidden	The client is authenticated but does not have permission to perform the requested operation, such as requesting a scope that is not allowed for the client
500	Internal server error	Contact the ASAP Support Team.

Appendix E: Verifying API Connectivity

This appendix provides details for being able to test connectivity with the ASAP Nexus API Gateways. The tests are illustrated in two ways:

1. Using the "curl" command from the command line
2. Using the "postman" application.

This is not a tutorial for either of those two testing methods and assumes the reader has familiarity with one, the other, or both.

OAuth Tests

Testing an API Connection with OAuth is a two-step process. First, you must access Authentication Server API and then you must Send a message to the API taking the authorization token returned in the response from the first API call and adding that to the header of the second message.

Note that this test would be applied to the cases where the ECC would be sending messages back to the ASAP Nexus using path **/asap/ecc** (Default Behavior (OAuth2)).

Step 1: Request Access Token

To access the token from the command line, enter the following:

```
curl --location 'https://2ty6uzxkda.execute-api.us-gov-west-
1.amazonaws.com/live/asap/ecc' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'grant_type=client_credentials' \
--data-urlencode 'scope=api/api-access' \
--data-urlencode 'client_id=29ub8hfcv1udcefs5uii3tp1lq' \
--data-urlencode
'client_secret=12d04j1241rv8f27g4gisc221ia901hitcnkkobkievl9ic780e0'
```

To request the code via Postman,

Search Postman Ctrl K | Invite | Upgrade

Overvi | POST Cer. | POST Mot | POST Sen | POST inbc | POST Cac | POST Cac | No environment

OAuth Token Requests / **inboundMotorolaTokenRequest** | Save | Share

POST https://asap-api-dev.auth-fips.us-gov-west-1.amazoncognito.com/oauth2/token **Send**

Params | Authorization | Headers (10) | Body | Scripts | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL

	Key	Value	Description	Bulk Edit
☑	client_id	Provided by MCP		
☑	client_secret	Provided by MCP		
☑	grant_type	client_credentials		
☑	scope	api/api-access		
	Key	Value	Description	

Body | Cookies (1) | Headers (14) | Test Results | 200 OK | 770 ms | 1.47 KB | Save Response

{} JSON | Preview | Visualize


```
1 {
2     "access_token":
        "eyJraWQiOiJpQVwvbzdwaWFUVWpoY0lKXC91aHFZSHUyZk5TR2Eyc3REZGp2K3ZFZFJPXC9NPSIsImFsZyI6IlJTMjU2In0.
        eyJzdWIiOiIyOXViOGhmY3YxdWRjZWZzNXVpaTN0cDFscSIsInRva2VuX3VzZSI6ImFjY2VzcyIsInNjb3BlIjoiYXBpXC9hc
        GktYWNjZXNzIiwiYXV0aF90aW1lIjoxNzQ4MDE3NzE0LCJpc3MiOiJodHRwczpcL1wvY29nbml0by1pZHAudXMtZ292LXdlc3
        QtMS5hbWF6b25hd3MuY29tXC91cy1nb3Ytd2VzdC0xX2d4RnE0cmpkRiIsImV4cCI6MTc0ODAyMTMxNCwiaWF0IjoxNzQ4MDE
        3NzE0LCJ2ZXJzaW9uIjoyLCJqdGkiOiIwZmRiMzI2Ny1jN2Q3LTQ4YWEtOTYzYi0xZjFiMGEwNGI0ZDAiLCJjbGllbnRfaWQi
        OiIyOXViOGhmY3YxdWRjZWZzNXVpaTN0cDFscSJ9.
        Kwwuii_4x1RNpw0fTU0NjApytOSEQHb8KYfhIGbYYOiTrxexiKyKbjKt_tIsOrghdcQsIPvuw89lIGH-KhQ7kq1Ao6XaFkus7
        zVuBx1AVtjIn0VEVgCrNDM49-ZETVh4oOpwluVM2wmXzXXlsqzMbvbPuGFwPqeekKCqwKKhdg7Ev5lESzQe3WAhvedqr9Los-
        gyhVMfEKGEmlLBZvLrEx3CoOXQO2PXFdwHT93TOlDbTJ5z9yFY2EnOMXt-zUFqE7f6dpnA9dExtgJTSASKcjaVBjBJ-uRewKu
        03rtg9StlwDhStWxjbP1mI6BD6-xLbxgBc4ChcYc4JOhUW1LiVA",
3     "expires_in": 3600,
4     "token_type": "Bearer"
5 }
```

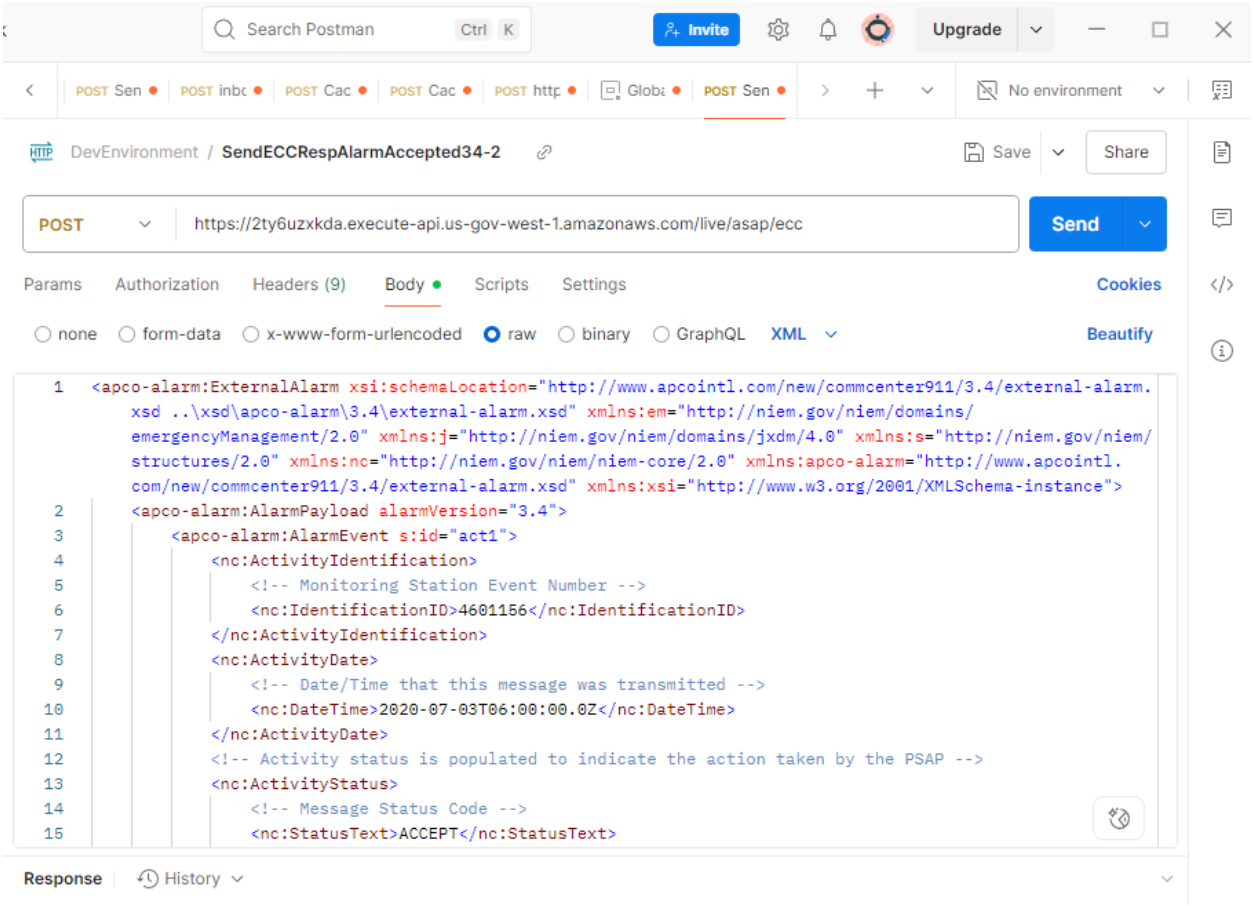

Postbot | Runner | Start Proxy | Cookies | Vault | Trash

Step 2: Send Message

Once the code has been received, the API is called to send the actual message. When the message is built, the value of the access_token in the response message is added to the header of the HTTP POST message. This is illustrated below with the command line.

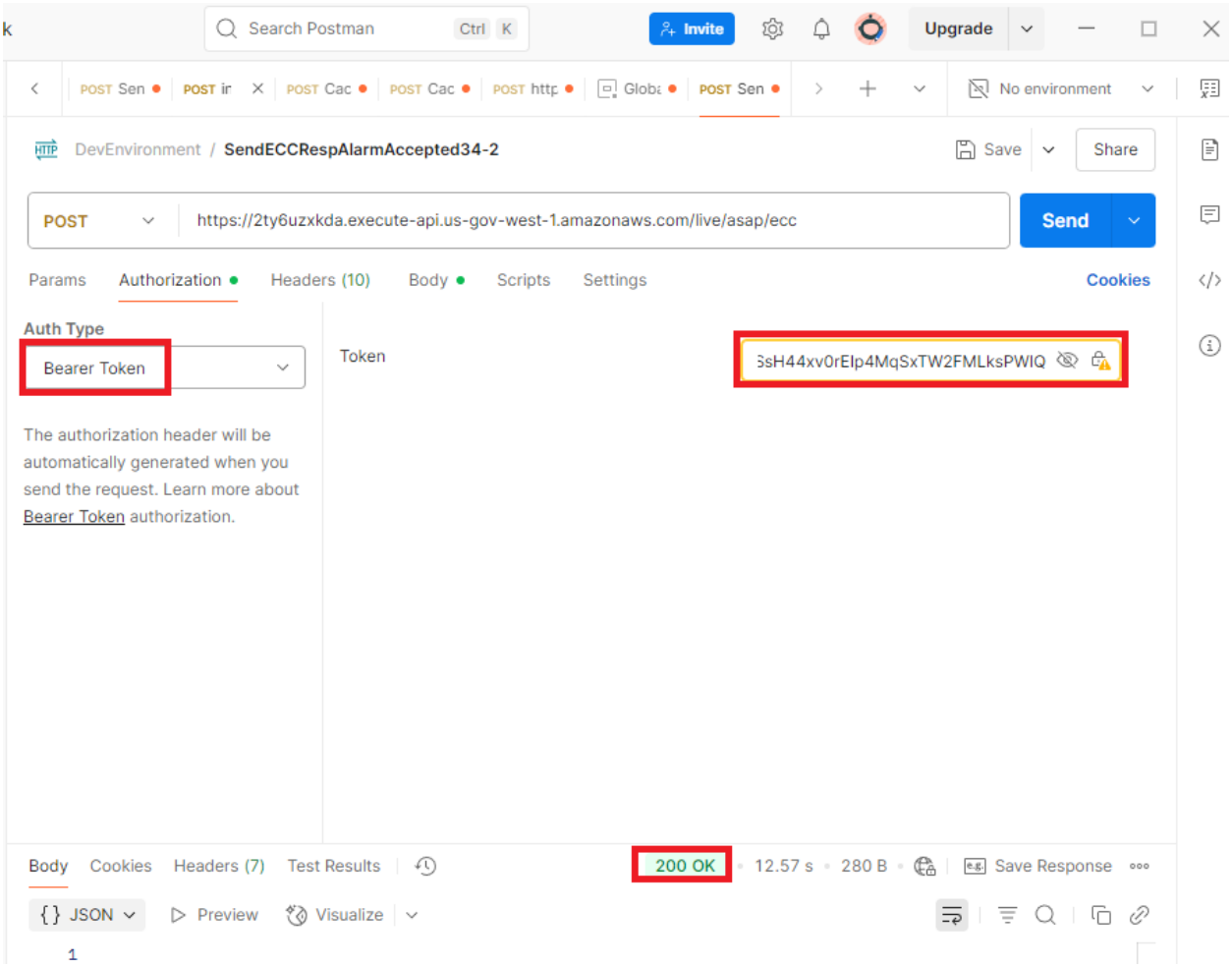
```
curl --location 'https://2ty6uzxkda.execute-api.us-gov-west-
1.amazonaws.com/live/asap/ecc' \
--header 'Authorization: Bearer <the value of auth_token from previous
response goes here' \
--header 'Content-Type: application/xml' \
--data '<apco-alarm:ExternalAlarm>...</apco-alarm:ExternalAlarm>'
```

And also below using postman by populating the body with the ASAP XML message,



And then adding the authorization header with the 'Auth Type' set to 'Bearer Token' and the token received in the token access response placed in the Token text box and sending the

message.



The response of 200 is received indicating successful submission.

MTLS Tests

The MTLS tests can be used to validate the monitoring center API and also the ECC API.

Monitoring Center API

The monitoring center API is accessed using the path **'/asap/central-station'.** Testing is somewhat convoluted in that it requires a fictitious (or real) host name added to the testing side DNS Server and associated with one or more IP addresses. The hostname is then configured into the ASAP Nexus using the admin portal. This is required because the API performs a secondary authorization verifying that the originating IP address presenting the certificate is really associated with the host. Once configured, the following command can be entered.

```
curl https://api-test.dsis-mcp.com/asap/central-station \
-d '<apco-alarm:ExternalAlarm>...</apco-alarm:ExternalAlarm>' \
-H "Content-Type: application/xml" -v \
--cert-type P12 \
--cert dev.mcp.cert.pfx:<password goes here> \
--cacert chain.pem
```

Of note is the use of the verbose option (-v) which displays the mtls handshake.

Also note that because the path for '/asap/central-station' performs the IP address validation, postman cannot be used to verify because it makes use of the cloud and the originating IP addresses are unpredictable, meaning a DNS server can't really be configured easily.

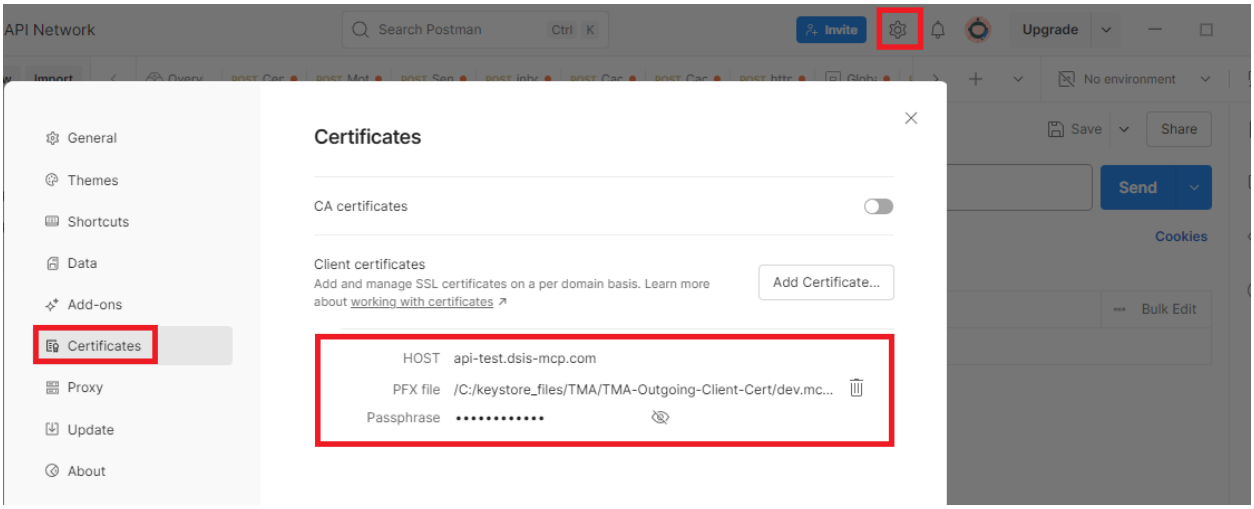
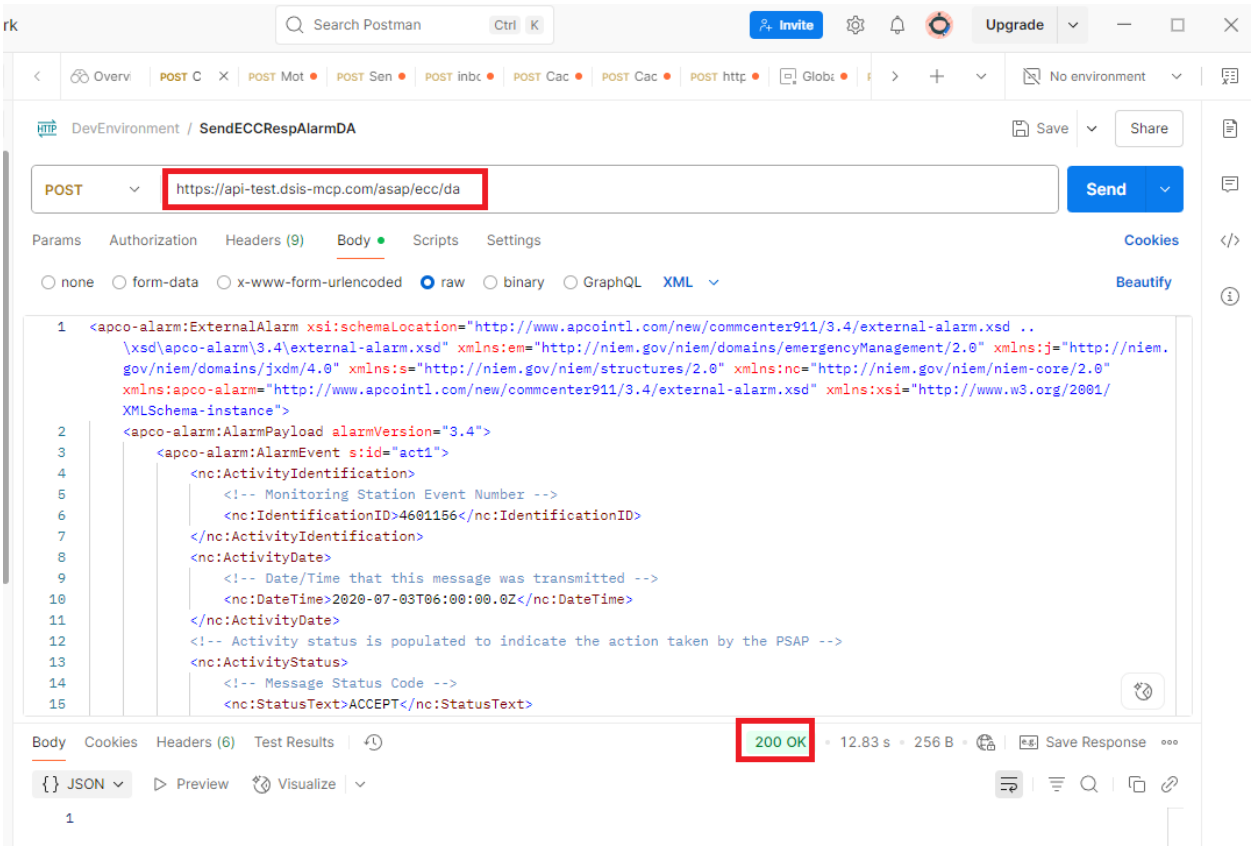
Note that this example shows the use of PFX certificates. Consult the curl command help pages for usage of the differ certificate types.

MTLS ECC API

The ECC API using MTLS authentication is accessed using the path **'/asap/ecc/da'**. While most ECCs use OAuth, there is at least one requiring the use of MTLS. Testing for this interface is somewhat easier than the API for monitoring centers.

The curl command can be executed as described in the previous section or, you can use postman as follows.

First, the API URL needs to be assigned a client certificate that has been added into the ASAP Nexus configuration via the admin portal.



This is performed via the settings/Certificates. The Host, the certificates and the certificate pass key must be provided. This example shows the use of a pfx certificate. Other options are available and can be found in the postman documentation.